EXPERIMENT – 4

Title: UDP Socket Programming

Objective:

Communicating between two computers located at different places and connected via network.

Software / Tools Required

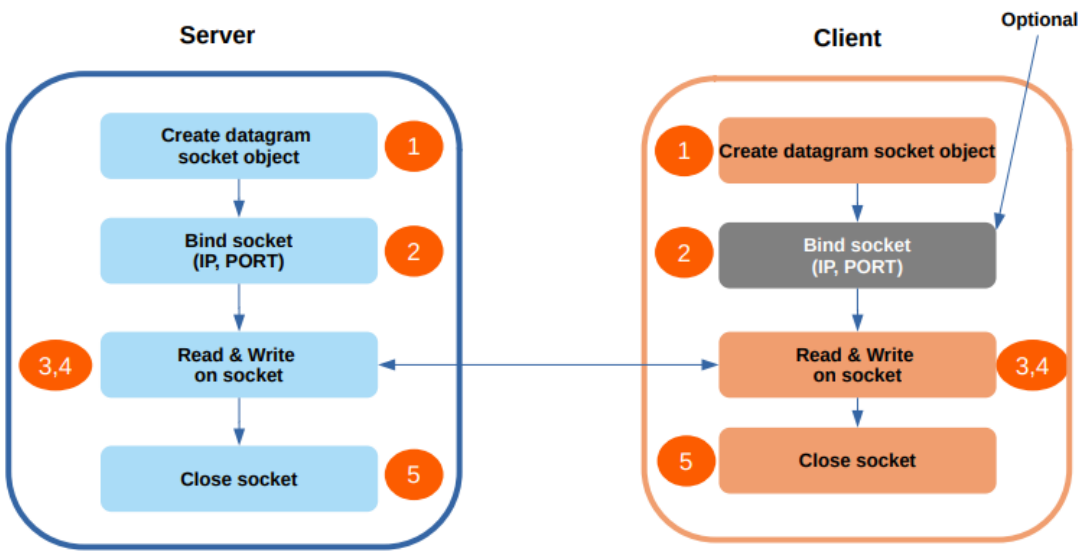
- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

UDP (User Datagram Protocol) is a connectionless protocol. It does not establish a proper connection like TCP. Data is sent in the form of datagrams. It is faster but less reliable because there is no guarantee of delivery.

In this experiment, we created a UDP server and UDP client using Python. The server generates random numbers and sends them to the client. The client receives the data and sends acknowledgement back. If client sends "close", the server closes the connection.

UDP Socket Flow:



Procedure / Algorithm

Server Side

1. Import required modules.
2. Create UDP socket.
3. Bind the socket with IP and Port.
4. Wait for client request.
5. Generate random number and send to client.
6. Receive acknowledgement from client.
7. If client sends “close”, terminate connection.

Client Side

1. Import required modules.
2. Create UDP socket.
3. Send initial message to server.
4. Receive data from server.
5. Send “+ACK” after receiving data.
6. If user enters ‘y’, send “close” and terminate.

Program Code

Server Program:

```python
import netpack as npk
import random


PORT=9999
server_ip="127.0.0.1"
ADDR=(server_ip, PORT)

# Step-1
server_socket=npk.create_udp_socket()

# Step-2
npk.start_udp_server(server_socket, ADDR)

# Step-3
print("[WAIT] Server waits for client")
(data, addr)=npk.read_data_from_udp_client(server_socket)
print('[CONNECTED] Client with IP : {} & PORT : {}'.format(addr[0], addr[1]))

while True:
    val=random.randint(0,100)
    npk.send_data_to_udp_client(server_socket, val, addr)
    print("[SEND] Data {}".format(val))

    (data, addr)=npk.read_data_from_udp_client(server_socket)
    if data=="close":
        npk.close_socket_connection(server_socket)
        print("[=] UDP Client requesting to close connection")
        break
    else:
        print("[RECEV] {}".format(data))
```

Client Program:

```python
import netpack as npk
import random


PORT=9999
client_ip="127.0.0.1"

server_ip=client_ip
ADDR=(server_ip, PORT)

# Step-1
client_socket=npk.create_udp_socket()

# Step-2
npk.send_data_to_udp_server(client_socket, "Hello", ADDR)
print('[CONNECTED] Server with IP : {} & PORT : {}'.format(ADDR[0], ADDR[1]))

while True:
    (data, ADDR)=npk.read_data_from_udp_server(client_socket)
    print('[=] Message from server : {}'.format(data))

    flag=input('Close connection y/n : ')
    if flag=='y' or flag=='Y':
        npk.send_data_to_udp_server(client_socket, "close", ADDR)
        npk.close_socket_connection(client_socket)
        print("[=] Requested for closing connection")
        break
    else:
        npk.send_data_to_udp_server(client_socket, "+ACK", ADDR)
```

Output:

Server Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-4$ python3 4_UDP_server.py
[WAIT] Server waits for client
[CONNECTED] Client with IP : 127.0.0.1 & PORT : 41788
[CONNECTED] Client with IP : 127.0.0.1 & PORT : 41788
[SEND] Data 21
[RECEV] +ACK
[SEND] Data 21
[RECEV] +ACK
[RECEV] +ACK
[SEND] Data 16
[RECEV] +ACK
[SEND] Data 56
[RECEV] +ACK
[SEND] Data 85
```

Client Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-4$ python3 4_UDP_client.py
[CONNECTED] Server with IP : 127.0.0.1 & PORT : 9999
[CONNECTED] Server with IP : 127.0.0.1 & PORT : 9999
[=] Message from server : 21
Close connection y/n : n
[=] Message from server : 21
Close connection y/n : n
Close connection y/n : n
[=] Message from server : 16
Close connection y/n : n
[=] Message from server : 56
Close connection y/n : n
[=] Message from server : 85
Close connection y/n :
```

Result

UDP client and server communication was successfully implemented and data was exchanged properly.

Conclusion

From this experiment, I understood how UDP works without connection setup. Data can be sent and received using socket programming, but reliability depends on application logic.